CIT4001 SOFTWARE IMPLEMENTATION AND TESTING

Software development and implementation report

PracticeChart EHR

University of Technology, Jamaica

Group 2

This document describes the implemented course application and the evidence recorded for it. It uses fictional patient data. Human training and user acceptance remain pending.

Public course copy. Student identification numbers and private course instruments are excluded.

Prepared for group review. This is not a record of clinical approval, a live pilot, completed training, or completed acceptance testing.

Introduction

PracticeChart is a working EHR course application for a private medical practice. It replaces the case study's paper record steps with role-controlled digital workflows. The intended users are doctors and front desk staff. Development, tests, screenshots, and the public demo use fictional records only. No live practice rollout, clinical approval, legal compliance, completed training, or human acceptance is claimed.

The project brief defines the EHR business scope. The assignment guide defines this report and the two manuals. The syllabus requires implementation, testing, metrics, maintenance, configuration management, and team participation. The old application and earlier change record were deleted. Their code and test results were not reused as EHR evidence.

Scope and objectives

Support patient registration and retrieval, administrative updates, safe appointment scheduling, consultation records, diagnostic files, prescriptions, certificates, visit filing, manual invoices, and reports. Preserve earlier clinical entries and audit history. Provide an encrypted full backup and a measured restore test. Keep the local presentation data across restarts.

There is no patient portal, payment processor, AI clinical advice, biometric authentication, external medical practice connection, Ministry of Health connection, pharmacy integration, or laboratory integration. Email delivery is deferred. In-app reminders are included. Hardware purchasing and staff training are not represented as completed work.

Requirements and constraints

REQUIREMENTS.md records source references, conflicts, and decisions. The scanned requirements include unique usernames, role limits, audit history, daily backup, a three second login target, a two second patient retrieval target, 50 concurrent users, and a 99 percent operational availability target. The short tests measure the first three performance conditions locally. They cannot prove long-term availability or legal compliance.

Development used a Windows computer, Node.js, Docker Desktop, PostgreSQL 18, and Chromium. The local benchmark environment was AMD Ryzen 5 7535HS with Radeon Graphics, 12 logical CPUs, 31.2 GiB RAM, Node v26.12.1, and Next.js 16.4.3. The cloud demo uses the existing Vercel project, separate Neon EHR databases, and new private Blob paths. Free service limits remain a constraint.

Inputs include credentials, patient demographics, booking intervals, doctor-entered clinical data, diagnostic files, and invoice lines. Outputs include confirmation and error messages, schedules, patient history, prescriptions, medical certificates, visit summaries, attendance slips, invoices, audit history, and recovery evidence. A patient report and full EHR backup are different outputs.

Design and implementation process

The brief, assignment guide, syllabus, schedule, supporting lesson documents, and all 53 embedded images were reviewed. Requirements were separated from old-system interviews, lesson examples, draft algorithms, and unsupported statements. Source conflicts were recorded before implementation. The user selected manual invoices and in-app reminders.

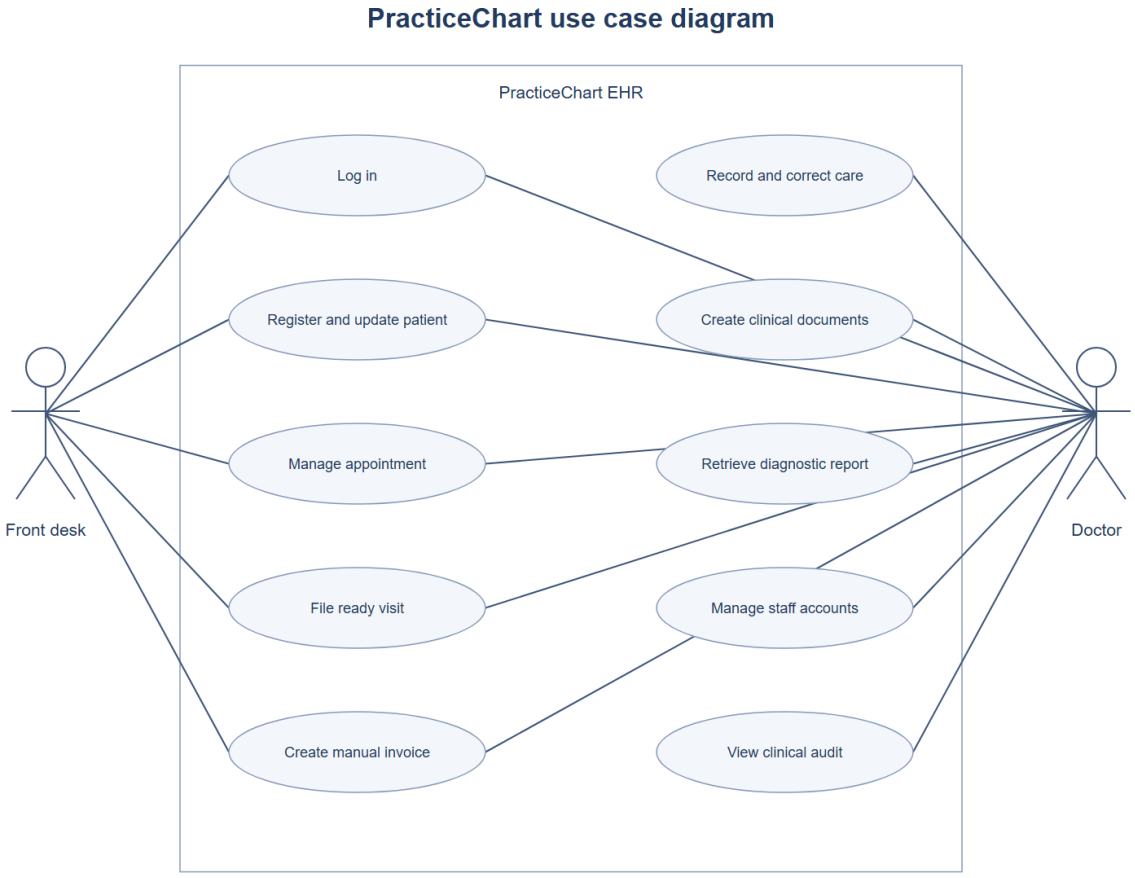
The implementation used small server services, validation schemas, explicit role checks, SQL migrations, database constraints, and a shared interface. The initial green sidebar design was rejected as too close to the

earlier application's appearance. It was replaced with ink-blue accents, warm light surfaces, top navigation, a patient worklist, and segmented record controls. No deleted application source or saved design reference was used.

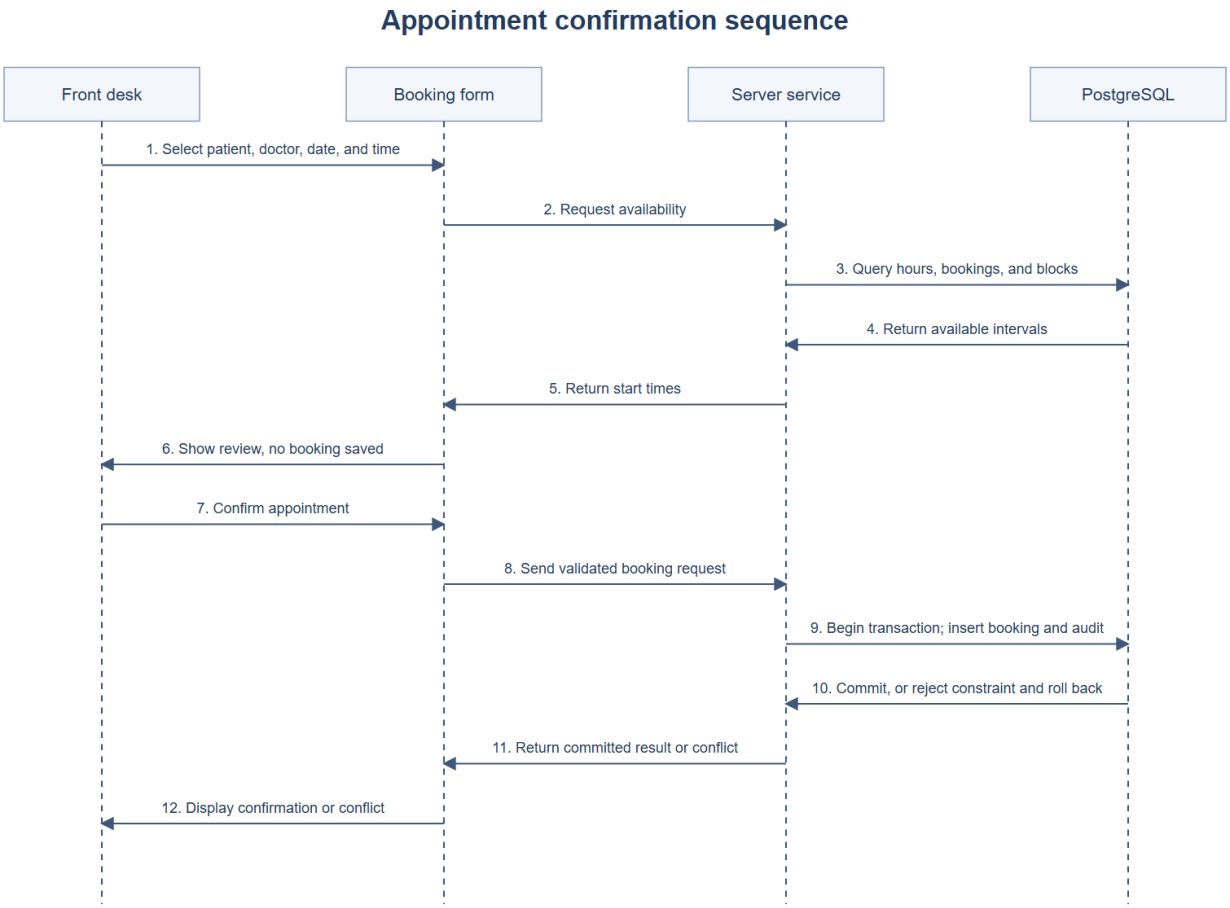
Coding and automated checks were performed with AI assistance in this workspace. These files do not assign module authorship to group members. The group must review the implementation, record each member's actual code contribution, and complete the human course tasks in GROUP_START_GUIDE.md.

Architectural design

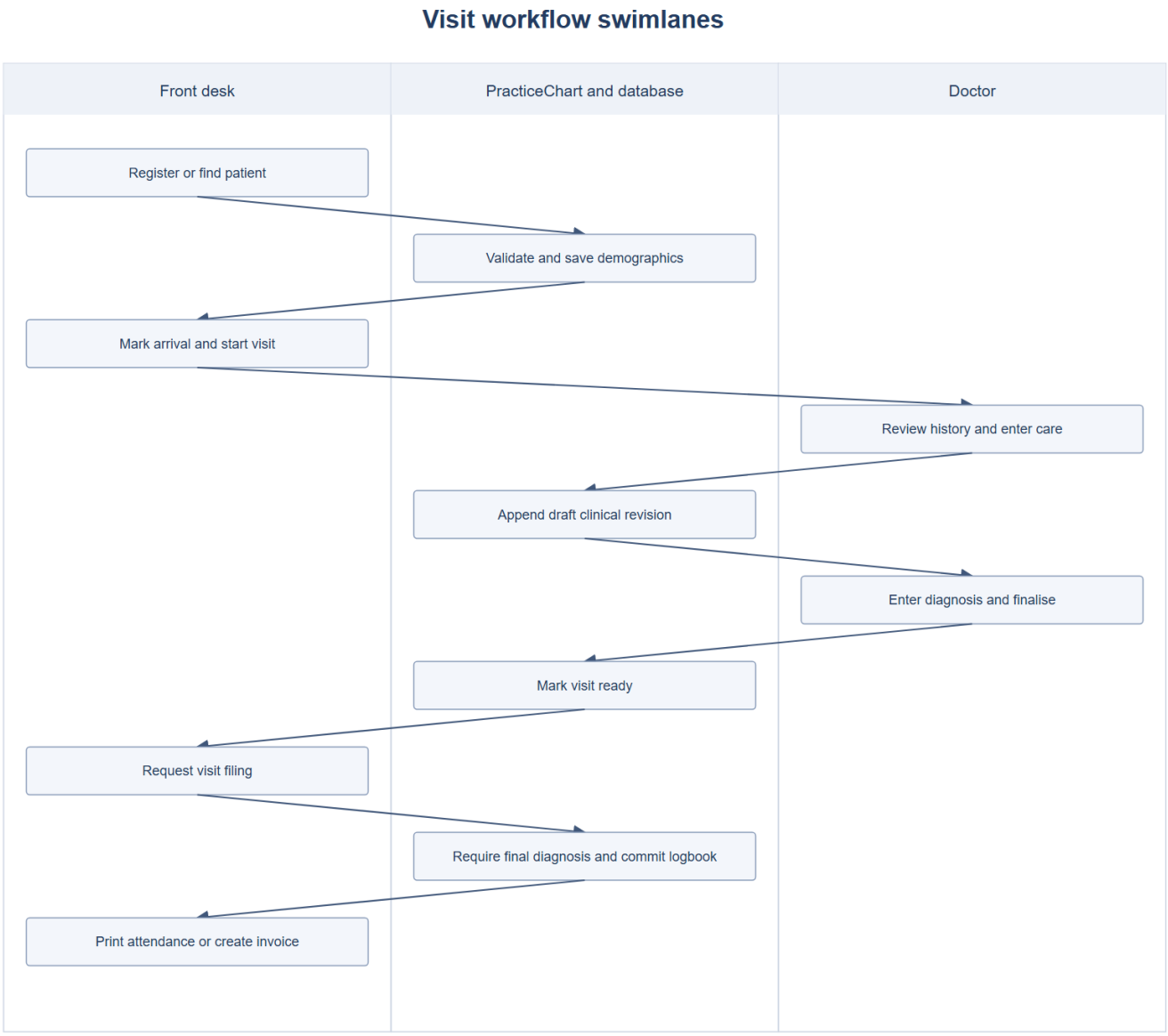
The browser uses Next.js pages and authenticated route handlers. Zod validates inputs. Drizzle services use PostgreSQL transactions. A restricted runtime database role cannot alter the schema or overwrite protected history. Clinical and audit changes commit together. Private files are streamed only after role checks. Workflow state contains IDs and result counts, not clinical bodies.



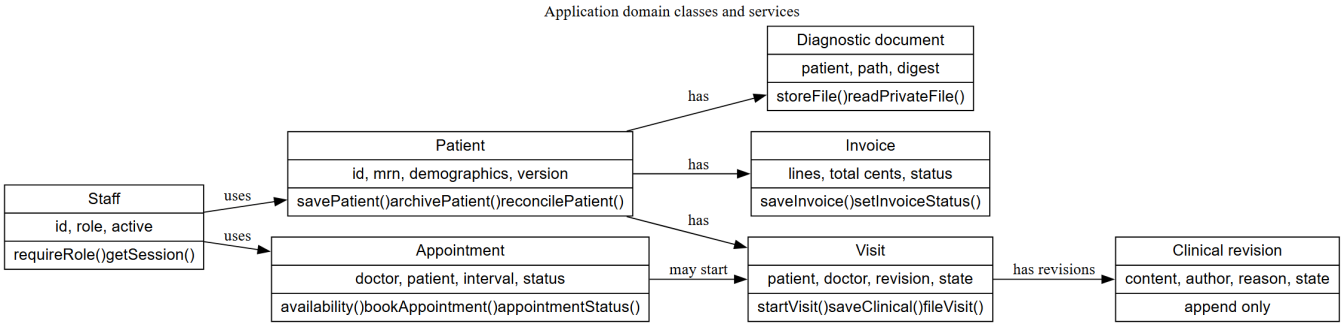
Use case diagram



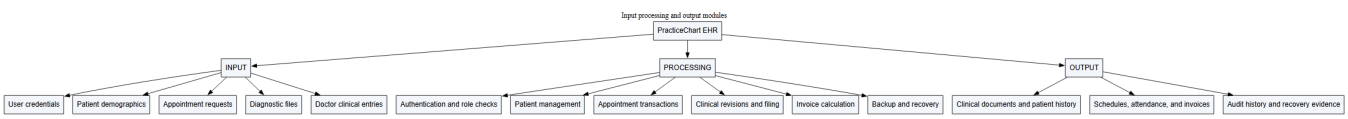
Sequence diagram



Swimlane diagram



Domain class and service diagram



Input processing and output modules

The class diagram represents domain models and service operations. The implementation uses TypeScript types, functions, and Drizzle table models. It does not pretend that every domain model is implemented as an

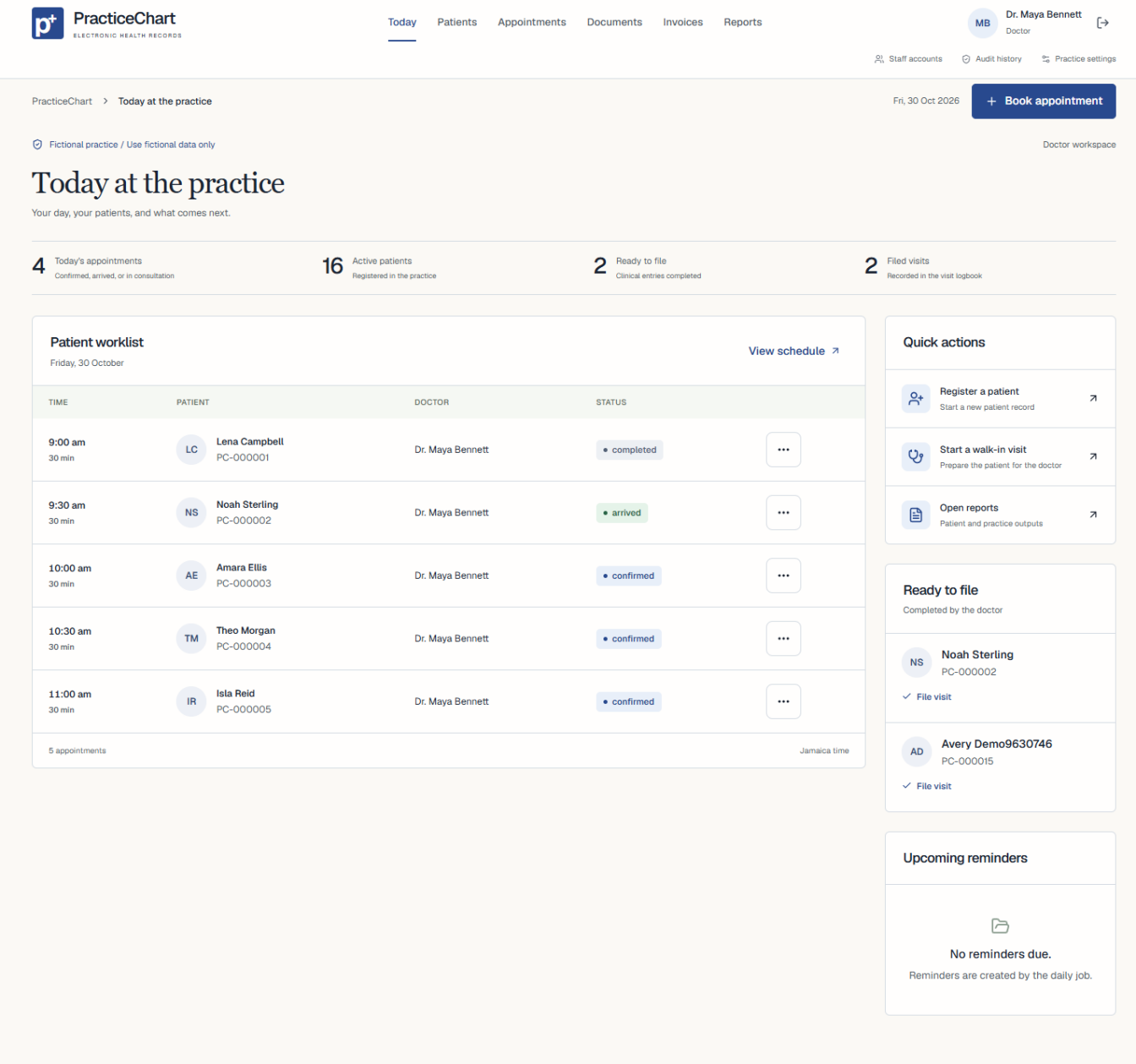
object-oriented class.

Procedural design

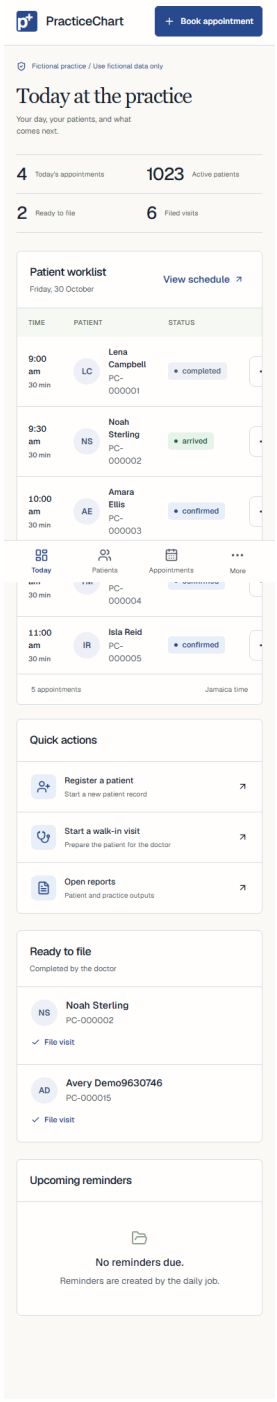
MODULES.md contains source-linked processing narratives and graph instructions for 26 functions. CONTROL_FLOW.md contains the three complexity calculations, planar face boundaries, and independent syntactic candidates. TEST_PLAN.md maps feasible state and input cases to executable tests. Database constraints are tested through PostgreSQL, rather than assigned invented JavaScript branch counts.

Interface design

The Today page leads with a patient worklist. The patient record separates demographics, visits, doctor-only clinical entries, and private documents. Shared forms show labels, required fields, errors, busy states, and confirmation. Booking uses a review step before commit. Final clinical corrections require a reason. Reduced-motion preferences remove motion. The phone view uses a compact bottom navigation.



Doctor workspace



Phone workspace

Testing and actual results

Vitest recorded 74 passing tests and 0 failed tests. Playwright recorded 8 passing browser tests and 0 unexpected failures. The measured Vitest library coverage was 91.42% of statements and 81.08% of branches. This is not a claim of full path coverage.

Unit and database tests cover valid and invalid partitions, boundaries, session and account states, simultaneous bookings, stale changes, clinical corrections, document rules, and invoices. Complete browser tests cover both roles. Direct clinical and file routes are denied to the front desk. Axe recorded zero violations on the nine checked desktop pages and the checked emulated phone page. Automated scans do not replace a full accessibility review or a real phone test.

Performance

The fixture contained 1016 fictional patients and 5 clinical revisions. Each load ran for 30 seconds. At loads above one, each background session requested a patient search and detail, then waited one second. One browser session measured complete record display. The first display is reported separately from later displays. The first display is not a claim of a cold server start.

Active sessions	Seconds	Browser login ms	First record ms	Warm display p95 ms	API requests	Request errors
1	30	258.99	148.87	164.21	0	0
10	30	255.45	138.26	190.66	522	0
50	30	625.71	144.72	271.42	2818	0

All measured browser logins were below three seconds. All measured warm record displays were below two seconds. Only one browser login was measured at each load. The server was a local production build. PostgreSQL ran in Docker on the same host. These results do not prove equivalent cloud performance or 99 percent availability.

Recovery

The full encrypted EHR recovery bundle restored into an isolated local PostgreSQL 18 database. The actual result was passed. The test compared 16 tables and 2 uploaded documents. The measured restore phase took 1.029 seconds. It checked file hashes and bytes, MRN sequencing, immutable histories, and overlap rejection. RECOVERY.md explains key custody, daily scheduling, retention, and limits.

Defects found and corrected

Observed defect	Cause and correction	Retest
Local login rejected a valid page origin	Internal host differed from the request host. Check the actual request host.	Both role logins pass
Available times changed after a newer date was selected	A late response replaced newer state. Ignore superseded responses.	Booking and rescheduling pass
Booking confirmation rejected a returned time	PostgreSQL timestamp output needed ISO normalisation. Normalise before submission.	Confirmation workflow passes
Saved textarea label lookup was inconsistent	Default text entered the nested label text. Add explicit accessible names.	Clinical correction workflow passes
A container did not trust the cloud certificate chain	Its trust store differed from Node. Supply the trusted Node CA set and retain hostname verification.	Cloud backup and restore evidence

Implementation method and maintenance

Use a phased conversion with a parallel reference process if the practice later approves real deployment. Begin with registration and scheduling. Add clinical recording and controlled document outputs after training. This reduces the amount of change at one time. It also requires extra checks and a clear authoritative record. This is a proposed method, not a completed pilot.

Use versioned migrations, reviewed branches, a lockfile, regression tests, and a release record. Keep recovery keys and backups private. Review failed daily jobs. Check restore ability after schema or storage changes. Train staff before any real use. Keep a support contact and issue register.

Human testing and remaining work

No completed user acceptance, staff training, live pilot, or real-device testing is claimed. Blank forms and a training plan are supplied. Each member must complete and explain a module contribution. Confirm the actual group presentation slot and private submission cover details with the tutor.

References

University of Technology, Jamaica. (n.d.). Assignment guide [Course handout].

University of Technology, Jamaica. (n.d.). Software implementation and testing syllabus [Course outline].

University of Technology, Jamaica. (2026). Software implementation schedule 2026 to 2027 semester 1 [Course schedule].

Group 2. (2026). Software implementation group project semester 1 2026 to 2027 [Private project brief].

PostgreSQL Global Development Group. (2026). PostgreSQL documentation: Range types. https://www.postgresql.org/docs/current/rangetypes.html

Neon. (2026). Migrate data from Postgres with pg_dump and pg_restore. https://neon.com/docs/import/migrate-from-postgres

Vercel. (2026). Private storage. https://vercel.com/docs/vercel-blob/private-storage

Appendices

See REQUIREMENTS.md, TEST_PLAN.md, CONTROL_FLOW.md, MODULES.md, RECOVERY.md, the diagrams, safe evidence files, programme manual, user manual, source listing, and blank training and acceptance forms. Private interviews and course chats are excluded from the public repository.

Source authority

The private source brief is Software Implementation Group Project Sem1-26_27.md. Its 53 embedded images were extracted and inspected during analysis. Images 26 and 27 contain the high level requirements. Image 32 contains F1 to F9. Image 33 contains N1 to N10. Images 38 to 41 are draft design diagrams. Images 42 to 53 are draft flow graphs. The raw brief, interviews, chats, and scans are not published here.

Assignment_Guide.docx controls the course report structure. Software_ImplTest_Syllabus (2).doc describes coding, testing, metrics, maintenance, configuration management, and team work. The semester schedule gives presentation windows. Lesson slides, tutorial exercises, and past papers support test methods. The revision guide is supporting material, not a higher authority than the course documents.

The source contains a private interview and observation notes. Those records describe the old practice. They are not evidence that this application passed human testing. The observation times and source performance claims conflict. No measured improvement is claimed from them.

Traceability

ID	Source and requirement	Implementation	Verification
AUTH-01	N1; draft login; doctor and front desk login	Argon2id, hashed sessions, expiry, login limit	Authentication integration tests and both role browser logins
AUTH-02	N2; account administration	Server role checks, disabled accounts, last doctor guard	Denied direct URL tests, account and session tests
PAT-01	F1, F2, UC01; registration fields and validation	Patient form, Zod, SQL date rule, unique MRN sequence	Date and string boundaries; duplicate name and MRN tests
PAT-02	F3; patient ID and name search	Parameterised search, demographic updates, version check	Search browser flow and stale update race
PAT-03	F4; interim record	Marked provisional record and doctor-controlled linking	Reconciliation test; both MRNs retained
APT-01	High level requirements; appointment requests	Doctor hours, available slots, review and confirmation	Booking browser flow and hour boundaries
APT-02	Draft appointment functions	Reschedule, cancel, arrival, completion	Browser reschedule and cancel; rollback tests
APT-03	User requirement; concurrent overlaps	PostgreSQL GiST exclusions for doctor and patient	Simultaneous booking and adjacent interval tests
CLN-01	F5; doctor diagnosis and treatment	Structured doctor form and versioned clinical records	Draft, final, stale revision, and correction tests
CLN-02	F6; lab results	Manual lab text and private diagnostic files	Exact file byte retrieval and denied access tests
VIS-01	F7 to F9; file visit and logbook	Ready flag, diagnosis check, filed timestamp	Filing before and after doctor completion
OUT-01	Prescription draft routine	Exact doctor-entered medication output	Prescription storage test and printed browser output
OUT-02	Receipt draft routine	Clinical visit summary and separate attendance slip	Role tests and print workflows
OUT-03	Lab report requirement	Document list and authenticated private retrieval	Private report browser tests
OUT-04	Patient report routine	Patient history, revisions, medications, file index	Report API and print tests
OUT-05	Image 27; medical certificate	Doctor-authored certificate output	Browser and report tests
BILL-01	Image 27 and user decision; invoices	Manual JMD line charges; draft, issue, void	Integer-cent partitions and invoice browser flow
AUD-01	N3; activity history	Transaction audit events and append-only revisions	History update/delete rejection and correction tests
REC-01	N9; daily backup and recovery	Full encrypted database and file bundle	Actual isolated restore result in evidence

ID	Source and requirement	Implementation	Verification
JOB-01	Image 27; reminders	Daily idempotent in-app reminder workflow	Workflow and duplicate reminder tests
PERF-01	N1, N5, N6; login, retrieval, 50 users	Production build and bounded connection pool	Stated 1, 10, 50 session benchmark
UX-01	N8; common tasks in at most three steps	Patient directory, search, open record	Keyboard and complete browser workflows
SEC-01	N7; protection in transit and at rest	Cloud TLS, managed storage, private files, encrypted recovery	Access tests and provider configuration review
OPS-01	N4; 99 percent operational availability	Health route and failure responses	Target only; no long-term uptime claim
GOV-01	N10 and scope statements	Access controls and evidence limits	No legal or clinical approval claim

Conflicts and decisions

Conflict or unsupported statement	Decision
Lab output repeats DisplayReceipt	Retrieve the actual diagnostic file by patient-linked document ID.
Receipt lists no money, but another page asks for invoices	Keep the visit summary separate. Add manual invoices by user choice. No payment processing.
Receptionist narrative displays full clinical history	Front desk receives demographic and visit-state data only. Clinical access needs a doctor role.
Duplicate interim files conflict with unique patient records	Make interim creation explicit and mark it provisional. Doctor linking preserves both identifiers and histories. Never bypass an outage by making another record.
Filing calls the patient inactive or archived	Close the visit. Keep the patient available for future care. Patient archiving is a separate doctor action.
Draft code stores or compares plain passwords	Store Argon2id hashes. Derive the role from the server account.
Sequence drafts confirm registration before a database save	Show success only after the transaction commits.
Draft graphs claim all input functions have complexity 2	Generate graphs from the implemented functions. Record the graph convention and calculate all three methods.
Email confirmation and reminders need a delivery service	Use on-screen confirmation and in-app reminders. External email is deferred by user choice.
Original text claims delays; interview says no major delay	Measure this build. Do not claim a measured reduction from that source.
Dates include old milestones and August 2027 implementation	Use the current course schedule for planning. Confirm the group presentation slot with the tutor.
Hardware purchases and vendor support appear in the scope	Supply local and cloud setup requirements. Do not claim equipment was bought or vendor training occurred.
Source states compliance, cost recovery, and user training	Treat these as unverified goals. Supply controls, a training plan, and blank acceptance forms.

Fixed defaults

The practice time zone is America/Jamaica. Initial doctor hours are Monday to Friday, 08:00 to 17:00. Available start times use 30 minute steps. The booking form allows several visit lengths; the database still rejects overlaps. Appointments must be at least one minute and no more than four hours. There is no patient self-service portal. Local upload storage is private. Public demo uploads accept only the supplied fictional report.